

Промежуточный отчёт по лабораторной работе №5 по предмету “Технологии управления данными”

Текст задачи:

Лабораторная работа №5. Создание Процедуры наполнения Витрины данных.
Цель работы: разработать скрипт, или хранимую процедуру, которая обеспечит наполнение Витрины данных на основании детальных данных, имеющихся в Хранилище. Витрина данных наполняется данными за определенный период, например за пять недель выбранных пользователем.

Решение:

Функция `random_datetime` в скрипте из лабораторной №1 изменена так, чтобы размах даты был уже (5 недель от 29.09.2025 до 03.11.2025).

Revenue - доход филиала за неделю

ListedPriceSum - сумма цен (по ценнику) товаров, проданных филиалом за неделю

Работа с датой и временем реализована с помощью встроенной в Python библиотеки `datetime`

Неделя - полуинтервал [полночь понедельника; полночь понедельника следующей недели)

Агрегирование данных производится построчным суммированием произведений (цена товара * количество)

Также, в SQL-скрипт построения схемы витрины данных было добавлено `GENERATED ALWAYS AS IDENTITY` для автоинкремента первичных ключей.

Код скрипта (на Python):

```
import psycopg2
import datetime

def date_id(x):
    year = x.year
    month = x.month
    day = x.day
    return f'{year:04}{month:02}{day:02}'

db_config_warehouse = {
    "dbname": "warehouse_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": "5448"
}

db_config_mart = {
    "dbname": "mart_db",
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
```

```

    "port": "5449"
}

conn_warehouse = None
conn_mart = None

start = datetime.datetime.strptime('2025-09-29 00:00', '%Y-%m-%d %H:%M')
end = datetime.datetime.strptime('2025-11-02 23:59', '%Y-%m-%d %H:%M')
monday = start
try:
    conn_warehouse = psycopg2.connect(**db_config_warehouse)
    conn_mart = psycopg2.connect(**db_config_mart)
    with conn_warehouse.cursor() as cur_warehouse:
        with conn_mart.cursor() as cur_mart:
            while monday < end:
                sunday = monday + datetime.timedelta(days=7)
                cur_mart.execute("INSERT INTO FiscalWeek (BeginDate,
EndDate) "
                                "VALUES (%s, %s) "
                                "RETURNING WeekID",
                                (str(monday), str(sunday)))
                weekID = cur_mart.fetchone()[0]
                cur_warehouse.execute("""SELECT DISTINCT StoreID FROM
SoldItem""")
                storeIDs = cur_warehouse.fetchall()
                for storeID in storeIDs:
                    price = 0
                    listed_price = 0
                    cur_warehouse.execute("SELECT StoreName FROM StoreInfo
WHERE StoreID = %s",
                                           (storeID,))
                    store = cur_warehouse.fetchall()[0]
                    cur_warehouse.execute("SELECT si.ActualPrice,
si.ListedPrice, si.Amount "
                                           "FROM SoldItem si "
                                           "WHERE si.DateID >= %s AND si.DateID < %s AND
si.StoreID = %s",
                                           (date_id(monday), date_id(sunday), storeID))
                    ...
                    или альтернативно:
                    cur_warehouse.execute("SELECT si.ActualPrice,
si.ListedPrice, si.Amount "
                                           "FROM SoldItem si "
                                           "JOIN Receipt r on r.ReceiptID = si.ReceiptID "
                                           "WHERE r.CompletedDate >= timestamp %s AND
r.CompletedDate < timestamp %s AND si.StoreID = %s",
                                           (str(monday), str(sunday), storeID))
                    ...

```

```

        rows_info = cur_warehouse.fetchall()
        for row_info in rows_info:
            price += row_info[0] * row_info[2]
            listed_price += row_info[1] * row_info[2]
        cur_mart.execute("INSERT INTO WeeklyStoreReport (WeekID,
StoreName, Revenue, ListedPriceSum) "
                        "VALUES (%s, %s, %s, %s)",
                        (str(weekID), store, str(price),
str(listed_price)))
        monday += datetime.timedelta(days=7)
        conn_mart.commit()
except (Exception, psycopg2.DatabaseError) as error:
    if conn_mart:
        conn_mart.rollback()
finally:
    if conn_mart:
        conn_mart.close()
    if conn_warehouse:
        conn_warehouse.close()
print("All done!")

```